The Countdown Numbers Problem: a Canonical Formulation

October 2026

Abstract

In the numbers round of the television game *Countdown*, contestants combine six dealt numbers with the four arithmetic operations to reach a three-digit target. We give a precise mathematical definition of the problem solved by the implementations in the accompanying repository [3]. The definition goes beyond "reach the target": it fixes the rules for intermediate values, an objective that prefers exact and then simpler answers, and a total tie-breaking order that makes the answer, including its printed text, a function of the input. We prove that the definition is well posed, show that it agrees with the staged "exact first, otherwise closest" procedure the solvers carry out, and record the structural facts the fast solvers rely on. We then describe the algorithms the solvers use and prove them correct, from the sequential reference to massively parallel GPU versions, and close with statistics and timings for every implementation on a 10,000-round benchmark set.

1 Introduction

A Countdown numbers round deals a *pool* of six positive integers and a *target* between 100 and 999. Using each number at most once, together with addition, subtraction, multiplication and division, players try to make the target. Every intermediate result must be a positive integer, and not every number has to be used. If the target cannot be reached, the closest attainable value wins.

The repository [3], at `https://github.com/ZippyManaic/countdown`, contains solvers for this game in more than thirty languages, from x86-64 assembly and line-numbered BASIC to CUDA, plus seven versions that share each round across several threads. They are required to produce *byte-identical* output, so the problem they solve has to be defined down to which of several equally good expressions is printed and how it is bracketed. This paper gives that definition. Section 2 describes the inputs. Section 3 defines legal expressions and the values they reach. Section 4 defines the objective and Section 5 the canonical answer. Section 6 collects the properties that efficient solvers exploit, Section 7 gives the algorithms with their correctness proofs and Section 8 reports measurements of the problem and Section 9 the timings of every implementation. Section 10 compares each language's straight port with its optimised version, Section 11 shares a single round across CPU threads and Section 15 records how the implementations were written. Hutton [1] gave an earlier formalisation that lists every solution. Here the task is to choose exactly one.

2 Instances

Definition 2.1 (Instance)**.** An *instance* is a pair (t, p) in which the *target* $t \in \mathbb{N}_{\geq 1}$ is a positive integer and the *pool* $p = (p_0, \ldots, p_{n-1})$ is a finite sequence of $n \geq 1$ positive integers. Write $[n] = \{0, \ldots, n-1\}$ for the set of *positions*.

The pool is a *sequence* rather than a multiset. Positions distinguish equal numbers, and Section 5 shows that the order of the pool can change the printed answer. It cannot change how good that answer is.

Definition 2.2 (Subsets and masks)**.** For $S \subseteq [n]$ let $\mu(S) = \sum_{i \in S} 2^i$ be its *mask*. The map μ is a bijection from subsets of $[n]$ to $\{0, \dots, 2^n - 1\}$, and we freely identify a subset with its mask. Write $|S|$ for the number of positions in S.

The game's distribution. A round of the game is dealt as follows. The target is uniform on $\{100, \dots, 999\}$. The number of *large* numbers ℓ is uniform on $\{1, 2\}$. The $6 - \ell$ *small* numbers are drawn without replacement from a deck holding two cards of each of $1, \dots, 10$, and the ℓ large numbers are drawn without replacement from $\{25, 50, 75, 100\}$. The pool lists the small numbers followed by the large ones. The solvers accept any pool with $1 \le n \le 6$, and the definitions below hold for every n.

Proposition 2.3. *Up to order, the game can deal exactly* 9,498 *distinct pools, so it has* $9{,}498 \times 900 = 8{,}548{,}200$ *distinct instances.*

Proof. A multiset of k small numbers using each of $1, \dots, 10$ at most twice corresponds to the coefficient of x^k in $(1 + x + x^2)^{10}$. That coefficient is 615 for $k = 4$ and 1,452 for $k = 5$. Two large numbers can be chosen in $\binom{4}{2} = 6$ ways and one in 4 ways. The large and small numbers never coincide, so the count is $6 \cdot 615 + 4 \cdot 1{,}452 = 9{,}498$. □

3 Expressions and reachable values

Definition 3.1 (Legal operations)**.** For integers $x \ge y \ge 1$, the *legal operations* on the ordered pair (x, y) are listed below in their *operator order*. Each one is legal only when its condition holds.

rank	operator	result	condition
0	$+$	$x + y$	always
1	$-$	$x - y$	$x > y$
2	$\times$	$x \cdot y$	$y \neq 1$
3	$\div$	x/y	$y \neq 1$ and $y \mid x$

Write $\mathrm{Ops}(x, y)$ for the list of legal pairs $(r, \circ)$ of result and operator. Every result is a positive integer.

Operations always take the larger operand first, so subtraction never goes negative and division never produces a fraction below one. The game's rules additionally allow $x - x = 0$, $x \times 1$ and $x \div 1$. Proposition 3.6 shows that excluding them never changes how good the best answer is.

Definition 3.2 (Expression)**.** An *expression* over p is a finite binary tree T with a value $\mathrm{val}(T)$ and a leaf set $\mathrm{lv}(T) \subseteq [n]$, built by two rules:

(i) for each $i \in [n]$ the leaf $\langle i \rangle$ is an expression with $\mathrm{val} = p_i$ and $\mathrm{lv} = \{i\}$;

(ii) if T_1 and T_2 are expressions with $\mathrm{lv}(T_1) \cap \mathrm{lv}(T_2) = \emptyset$ and $\mathrm{val}(T_1) \ge \mathrm{val}(T_2)$, and $(r, \circ) \in \mathrm{Ops}(\mathrm{val}(T_1), \mathrm{val}(T_2))$, then the node $(T_1 \circ T_2)$ is an expression with $\mathrm{val} = r$ and $\mathrm{lv} = \mathrm{lv}(T_1) \cup \mathrm{lv}(T_2)$.

Disjointness of leaf sets says that each dealt number is used at most once. The number of numbers an expression *uses* is $|\mathrm{lv}(T)|$.

Definition 3.3 (Reachable values)**.** For non-empty $S \subseteq [n]$, let

$$V(S) = \{\mathrm{val}(T) : T \text{ an expression with } \mathrm{lv}(T) = S\}.$$

A *split* of S is an unordered pair $\{L, R\}$ of non-empty sets with $L \cup R = S$ and $L \cap R = \emptyset$. Because μ is injective, each split has a unique representative with $\mu(L) < \mu(R)$.

Theorem 3.4 (Subset recursion)**.** $V(\{i\}) = \{p_i\}$*, and for* $|S| \geq 2$

$$V(S) = \bigcup_{\{L,R\} \text{ split of } S} \; \bigcup_{a \in V(L), b \in V(R)} \{\, r : (r, \circ) \in \mathrm{Ops}(\max(a,b), \min(a,b)) \,\}.$$

Proof. A tree whose leaf set is a singleton must be that leaf, which gives the base case. For $|S| \geq 2$ a tree T with $\mathrm{lv}(T) = S$ is a node $(T_1 \circ T_2)$. Its children's leaf sets form a split of S, and their values lie in $V(L)$ and $V(R)$, so $\mathrm{val}(T)$ belongs to the right-hand side. Conversely, take $a \in V(L)$ and $b \in V(R)$ for a split, witnessed by trees A and B. Put the one with the larger value first, taking A when $a = b$. Then each legal operation builds a node whose value is the corresponding result, and its leaf set is S. □

Proposition 3.5 (Value bound)**.** *Every* $v \in V(S)$ *satisfies* $1 \leq v \leq \prod_{i \in S}(p_i + 1) - 1$.

Proof. Use induction on $|S|$, where the base case is $p_i \leq (p_i + 1) - 1$. For $x \geq y \geq 1$ each legal result is at most $\max(x + y, xy) \leq xy + x + y = (x + 1)(y + 1) - 1$, and the bounds multiply over a split. □

For a game pool this is below $101 \cdot 76 \cdot 11^2 \cdot 10^2 < 2^{27}$, so every value fits in 32 bits; Section 13 revisits this for pools with more large numbers. Several solvers size a bitset of seen values by this bound.

Proposition 3.6 (Restricted rules lose nothing)**.** *Allow also the results* $x - x = 0$*,* $x \times 1$ *and* $x \div 1$*, and let* $d^\dagger$ *and* $k^\dagger$ *be the optimal distance and number count of Definition 4.1 under these looser rules. If* $t \geq 1$ *then the restricted rules attain the same* $d^\dagger$ *with the same* $k^\dagger$*.*

Proof. It suffices to show that any loose expression using one of the extra moves can be replaced by one that is no worse in distance and strictly better in distance or count. A node $T_1 \times 1$ or $T_1 \div 1$ can be replaced by T_1, which has the same value and one leaf fewer, and the replacement propagates up unchanged. Next consider a zero. Its parent combines it with some $w \geq 0$ through $w + 0$, $w - 0$ or $w \times 0$, giving w or 0. In the first case the zero subtree can be deleted, which keeps the value and uses fewer leaves. Otherwise zero propagates to the root, so the whole expression has distance t. Take a lowest zero node $x - x$ with $x \geq 1$. If $x \neq 1$, replace it by $x \div x = 1$, which uses the same leaves. If $x = 1$, take either operand subtree, which has value 1 and fewer leaves. Either way a value of 1 is reached, at distance $t - 1 < t$. □

4 The objective

Definition 4.1 (Candidates and the objective)**.** The *candidates* of an instance are the pairs (S, v) with $\emptyset \neq S \subseteq [n]$ and $v \in V(S)$. Each candidate has the key

$$\kappa(S, v) = \big(\, |t - v|,\ |S|,\ \mu(S),\ v \,\big) \in \mathbb{N}^4,$$

and the *optimal candidate* (S^*, v^*) is the candidate with the lexicographically least key. Call $d^* = |t - v^*|$ the *optimal distance* and $k^* = |S^*|$ the *optimal count*.

The four components, in priority order, say: get as close to the target as possible, then use as few numbers as possible, then prefer the subset with the smallest mask, then the smaller value. The last component only separates $t - d$ from $t + d$, and it picks the value below the target.

Proposition 4.2. *The optimal candidate exists and is unique.*

Proof. There is at least one candidate, since $(\{0\}, p_0)$ qualifies, and there are finitely many, since each $V(S)$ is finite by Proposition 3.5. The key determines S through μ and also v, so distinct candidates have distinct keys. □

The solvers do not minimise κ directly. They follow the staged procedure below, which mirrors how the game is played.

Proposition 4.3 (Staged form)**.** *Let $F = [n]$ be the whole pool. The optimal candidate is produced by the following procedure.*

(a) If $t \in V(S)$ for some S, let k be the least such $|S|$. Return (S, t) for the least mask S of size k with $t \in V(S)$.

(b) Otherwise let (S_1, v_1) minimise κ over candidates with $S \neq F$, and let v_2 minimise $(|t - v|, v)$ over $v \in V(F)$. Return (F, v_2) if $|t - v_2| < |t - v_1|$, and (S_1, v_1) otherwise.

Proof. Case (a) is the definition restricted to distance 0. In case (b), F is the only subset of size n, so a candidate on F has a count larger than every other candidate. A candidate on F therefore beats (S_1, v_1) under κ exactly when it is strictly closer. Among candidates on F, the mask and size are fixed, so κ reduces to $(|t - v|, v)$. When $n = 1$ the set of candidates with $S \neq F$ is empty and the procedure returns the single number. □

Stage (b) only ever needs the closest values of $V(F)$, never the whole set. This is why solvers can stream the full pool's results without storing them.

5 The canonical answer

Many expressions can share the optimal value and subset. For example $4 + 2$ and 2×3 might both give 6 from different subsets, and a single subset typically has many trees per value. To make the output a function of the input we fix one tree, the first derivation in a prescribed enumeration order.

Definition 5.1 (Split order)**.** For $|S| \geq 2$ the *split order* $\Sigma(S)$ is the sequence of sets L such that $\emptyset \neq L \subsetneq S$ and $\mu(L) < \mu(S \setminus L)$, listed in *decreasing* order of $\mu(L)$. Each split $\{L, S \setminus L\}$ appears exactly once. Write $\sigma_S(L)$ for the position of L in this sequence.

This is the order produced by the standard sub-mask loop $L \leftarrow (S - 1)\ \&\ S$, then $L \leftarrow (L - 1)\ \&\ S$ repeatedly, keeping only $L < S \oplus L$.

Definition 5.2 (Canonical derivation)**.** Let $|S| \geq 2$ and $v \in V(S)$. A *derivation* of v from S is a tuple $(L, a, b, \circ)$ with $L \in \Sigma(S)$, $R = S \setminus L$, $a \in V(L)$, $b \in V(R)$ and $(v, \circ) \in \mathrm{Ops}(\max(a, b), \min(a, b))$. The *canonical derivation* $\omega(S, v)$ is the derivation that minimises

$$\big(\, \sigma_S(L),\ a,\ b,\ \mathrm{rank}(\circ) \,\big)$$

lexicographically, where the rank is the one in Definition 3.1.

The canonical derivation exists because $v \in V(S)$ and Theorem 3.4 guarantee at least one derivation. It is unique because the key, together with S, determines the tuple. In implementation terms it is the *first* way of making v found when the splits are visited in order, each half's values in ascending order and the operators in operator order.

Definition 5.3 (Canonical tree)**.** Define $\tau(S, v)$ for every candidate by recursion on $|S|$. If $S = \{i\}$, then $\tau(S, v) = \langle i \rangle$. Otherwise let $\omega(S, v) = (L, a, b, \circ)$ and $R = S \setminus L$, and set

$$\tau(S, v) = \begin{cases} \big(\tau(L, a) \circ \tau(R, b)\big) & \text{if } a \geq b, \\ \big(\tau(R, b) \circ \tau(L, a)\big) & \text{if } a < b. \end{cases}$$

The recursion terminates because $|L|, |R| < |S|$. When the operands are equal the half with the smaller mask is written first. Every subtree of a canonical tree is itself canonical for its own subset and value. This is what lets a solver store a single origin per reachable value.

Definition 5.4 (Rendering)**.** The text $\rho(T)$ of an expression is defined by $\rho(\langle i \rangle) =$ the decimal digits of p_i, and

$$\rho\big((T_1 \circ T_2)\big) = \texttt{(}\ \rho(T_1)\ \texttt{␣}\ s(\circ)\ \texttt{␣}\ \rho(T_2)\ \texttt{)},$$

where $s(+) = \texttt{+}$, $s(-) = \texttt{-}$, $s(\times) = \texttt{*}$ and $s(\div) = \texttt{/}$, and juxtaposition is concatenation. The *display text* $\hat{\rho}(T)$ removes the first and last characters of $\rho(T)$ when T is a node and equals $\rho(T)$ when T is a leaf.

Every node is fully bracketed, with the larger operand on the left, so no precedence conventions are needed to read the text back. Only the outermost brackets are dropped.

Definition 5.5 (The Countdown problem)**.** Given an instance (t, p), compute

$$\mathrm{Solve}(t, p) = \big(\, v^*,\ \hat{\rho}(\tau(S^*, v^*))\,\big),$$

where (S^*, v^*) is the optimal candidate of Definition 4.1.

Example 5.6. For $t = 125$ and $p = (1, 5, 7, 3, 75, 50)$ no single number equals 125, but $75 + 50$ does, so $k^* = 2$ and Solve $= (125, \texttt{75 + 50})$.

Example 5.7. For $t = 423$ and $p = (6, 2, 2, 1, 1, 25)$ the target needs all six numbers:

$$\mathrm{Solve}(t, p) = \big(423,\ \texttt{(25 * ((6 * (2 + 1)) - 1)) - 2}\big).$$

Example 5.8. For $t = 431$ and $p = (3, 2, 3, 4, 2, 25)$ the target is unreachable. Both 430 and 432 lie at distance 1. The key's value component would favour 430, but 430 needs all six numbers while 432 needs five, so the count decides: $\mathrm{Solve}(t, p) = \big(432,\ \texttt{((25 * 3) - 3) * (4 + 2)}\big)$.

Remark 5.9 (Invariance)*.* Permuting the pool permutes the positions and preserves every $V(S)$ up to relabelling, so d^*, k^* and v^* can change only through the mask and value components of the key. The two *costs* (d^*, k^*) are invariant. The *display text* is not, because masks and split orders depend on positions. This is why the lookup-table solver, which answers for the sorted pool, matches the reference text on only 69% of rounds while never being worse.

Definition 5.10 (Equally good answers)**.** An answer (v, e) to (t, p) is *equally good* as $\mathrm{Solve}(t, p)$ when e is a legal expression over p with value v, $|t - v| = d^*$ and e uses k^* numbers.

The benchmark checker counts an answer as *identical* when it matches Solve byte for byte, as a *tie* when it is equally good, and as *wrong* otherwise. In practice the checker approximates legality by evaluating the text and comparing the value and the count of numbers.

6 Structure used by the solvers

Proposition 6.1 (Number of splits)**.** *A set of size $k \geq 2$ has $2^{k-1} - 1$ splits, and the pool as a whole has $\sum_k \binom{n}{k}(2^{k-1} - 1) = \frac{1}{2}(3^n - 2^{n+1} + 1)$ splits across all its subsets. For $n = 6$ this is* 301.

Proof. Ordered pairs (L, R) with both non-empty number $2^k - 2$, and each split counts twice. For the total, use $\sum_k \binom{n}{k}2^k = 3^n$. □

Proposition 6.2 (Smallest subsets first)**.** *If $t \in V(S)$ for some S with $|S| = k$, then $k^* \leq k$ and no subset of size greater than k affects the answer. In particular, a solver that builds V in increasing order of size can stop at the first size whose subsets contain the target. Among those subsets, the smallest mask is the answer.*

Proof. Any candidate with $|S| > k$ has a key with first component at least 0 and second component greater than k, so it loses to (S, t). The canonical tree of the answer only involves subsets of S^*. □

Testing whether $t \in V(S)$ does not require building $V(S)$. It is enough to look, for each split and each value on one side, for the few values on the other side that combine to t.

Lemma 6.3 (Partners)**.** *Let $a, b, t \geq 1$. Some legal operation on $(\max(a, b), \min(a, b))$ yields t if and only if b belongs to the set of positive integers among*

$$t - a, \qquad a - t, \qquad a + t, \qquad \tfrac{t}{a} \;\; (a \mid t,\ \min(a, \tfrac{t}{a}) \neq 1), \qquad \tfrac{a}{t} \;\; (t \mid a,\ \tfrac{a}{t} \neq 1), \qquad at \;\; (a \neq 1).$$

Proof. These are the solutions for b of $a + b = t$, $|a - b| = t$ with $a \neq b$, $ab = t$ with neither factor 1, and the quotient of the larger by the smaller equalling t with divisor not 1. Positivity of t gives $a \neq b$ in the subtraction cases. For the quotient $a/b = t$ the divisor is $b = a/t \leq a$. For $b/a = t$ it is a, and $b = at \geq a$. □

So $t \in V(S)$ can be decided with at most six membership queries per value on the smaller half of each split. The canonical derivation is the least $(\sigma_S(L), a, b, \text{rank})$ among the hits. The fast solvers also use this lemma recursively, to decide $t \in V(S)$ for $|S| = n$ without first building V for subsets of size $n - 1$.

Remark 6.4 (Hardness). With only addition allowed, deciding whether $d^* = 0$ is the Subset Sum problem, which is NP-complete when numbers are written in binary. The game avoids this by fixing $n = 6$ and keeping numbers small. Every instance is then a constant-size computation, and the interest lies in its constant factors.

7 Algorithms

This section describes how the implementations compute Solve and proves that they compute it exactly. Section 7.1 gives the reference algorithm that every byte-identical solver follows. Sections 7.2 and 7.3 cover the refinements used on CPUs and GPUs. Section 7.4 covers the two versions that return equally good rather than identical answers.

7.1 The reference algorithm

The reference algorithm stores, for every subset S it builds, the sorted list of $V(S)$ with one *origin* per value, recording the derivation that made it. Two procedures act on a subset. Build enumerates every derivation in canonical order and keeps the first one for each value. Find decides whether a value is in $V(S)$ without building $V(S)$, by scanning one half of each split and looking up the partners of Lemma 6.3 in the other.

Algorithm 1 Reference solver for Solve(t, p)

procedure Build(S)
  $W \leftarrow$ empty map from values to derivations
  for $L \in \Sigma(S)$ in order, with $R = S \setminus L$ **do**
    for $a \in V(L)$ ascending, $b \in V(R)$ ascending **do**
      for $(r, \circ) \in \mathrm{Ops}(\max(a,b), \min(a,b))$ in operator order **do**
        if $r \notin W$ **then** $W[r] \leftarrow (L, a, b, \circ)$
  store $V(S) \leftarrow$ the keys of W sorted ascending, each with its derivation

procedure Find(S, v)
  for $L \in \Sigma(S)$ in order, with $R = S \setminus L$ **do**
    $H \leftarrow \{(a, b, \circ) : a \in V(L),\ b \in V(R),\ (v, \circ) \in \mathrm{Ops}(\max(a,b), \min(a,b))\}$
    ▷ enumerate H by scanning the shorter of $V(L)$ and $V(R)$ and looking up partners
    if $H \neq \emptyset$ **then return** $(L, \min H)$
  return $\perp$

procedure Solve(t, p)
  for $i \in [n]$ **do** store $V(\{i\}) \leftarrow \{p_i\}$
  if $p_i = t$ for some i **then return** the leaf for the least such i
  for $k \leftarrow 2$ **to** n **do**
    for S with $|S| = k$ in increasing mask order **do**
      if Find$(S, t) \neq \perp$ **then return** $(t, \hat{\rho}(\tau(S, t)))$
    if $k < n$ **then**
      for S with $|S| = k$ **do** Build(S)
  $(S_1, v_1) \leftarrow$ least κ over all stored values of subsets $S \neq [n]$
  $v_2 \leftarrow$ least $(|t - v|, v)$ over results of the pairs of every split of $[n]$, streamed and not stored
  return $([n], v_2)$ if $|t - v_2| < |t - v_1|$, otherwise (S_1, v_1), rendered through the origins

Theorem 7.1 (Correctness)**.** *Algorithm 1 returns* Solve(t, p)*.*

Proof. Build keeps the canonical derivation. By induction on $|S|$ the stored lists are exactly the sets $V(L)$, sorted. Build(S) then visits every derivation of every $v \in V(S)$ in increasing order of the key $(\sigma_S(L), a, b, \mathrm{rank})$ of Definition 5.2, and it keeps the first, which is the least: this is $\omega(S, v)$. The set of keys is $V(S)$ by Theorem 3.4.

Find returns the canonical derivation. The relation "some legal operation on a and b yields v" is symmetric in a and b, so scanning either half finds the same set H, and Lemma 6.3 lists, for each scanned value, every partner that could be in H. Splits are visited in order and the least element of H is taken, so the result is $\omega(S, v)$ whenever $v \in V(S)$, and $\perp$ otherwise.

The loop realises the objective. Searching size k needs only the lists for sizes below k, which have been built. The first size and least mask with a hit give case (a) of Proposition 4.3. If no size hits, every proper subset has been built, and the last three lines are case (b) verbatim.

Rendering. The stored origins are the canonical derivations, so following them from the answer reproduces $\tau(S^*, v^*)$, which is rendered as $\hat{\rho}$. □

Cost. Table 1 measures the work by subset size. Building size k forms every pair across every split of every k-subset, while searching forms only one scan of the shorter half of each split. At size five, searching scans 214 values and building forms 7,140 pairs, so the early exit of Proposition 6.2 matters far more than any constant factor. The search also replaces the build for the whole pool, where building would form 15,456 pairs.

k	pairs to build size k	legal results	values scanned to search size k
2	15	46	15
3	184	571	60
4	1,446	4,522	185
5	7,140	22,334	214
6	15,456	48,213	196

Table 1: Work per round summed over all subsets of size k, averaged over the first 300 benchmark rounds. Each scanned value costs at most six lookups.

7.2 Refinements on the CPU

The compiled implementations keep Algorithm 1 and change how BUILD removes duplicates and sorts. Each refinement is exact, so the output is unchanged.

Duplicates by bitset. Instead of a map, a bitset B over $\{0, \dots, M(S)\}$ records which values have been seen. Each legal result is written to the next output slot unconditionally, and the output pointer advances by one minus the old bit. The first derivation of each value is therefore the one kept, as before, with no branch to mispredict.

Sorting by rank. Once every result is in, the bitset itself is the sorted set.

Lemma 7.2 (Rank)**.** *Split B into 64-bit words $B_0, B_1, \dots$, and let $c_j = \sum_{i<j} \operatorname{popcount}(B_i)$. A value $v \in B$ belongs at index $c_{\lfloor v/64 \rfloor} + \operatorname{popcount}\big(B_{\lfloor v/64 \rfloor} \,\&\, (2^{v \bmod 64} - 1)\big)$ of the ascending list.*

Proof. The expression counts the members of B below v: whole words before v's word, then the lower bits of its own. □

This sorts the list in time linear in the number of values and words, with no comparisons. Sets with few values use insertion sort instead.

A tighter value bound. The bitset is sized by $M(S)$, where $M(\{i\}) = p_i$ and $M(S) = \max_{\{L,R\}} \max\big(M(L) + M(R),\ M(L)\,M(R)\big)$. Every $v \in V(S)$ satisfies $v \le M(S)$, since subtraction and division never exceed the larger operand and addition and multiplication are monotone. This is at most the bound of Proposition 3.5.

Division by reciprocal. Integer division is slow, so the solvers test exactness with a precomputed floating-point reciprocal.

Lemma 7.3 (Reciprocal division)**.** *Let $1 \le y \le x < 2^{31}$ and let q be $x \cdot \mathrm{fl}(1/y)$, computed in IEEE double precision and rounded to the nearest integer. Then $y \mid x$ if and only if $qy = x$.*

Proof. With unit roundoff $u = 2^{-53}$, the computed product differs from x/y by at most $(x/y)(2u + u^2) < 2^{31} \cdot 2^{-51} < \frac{1}{2}$. If $y \mid x$ then x/y is the nearest integer, so $q = x/y$ and $qy = x$. If $y \nmid x$, no integer q satisfies $qy = x$. □

Values stay below 2^{27} (Proposition 3.5), so the lemma applies with room to spare. An inexact division writes 0, whose bit is set before building starts, so the bitset drops it without a branch.

Known larger operand. Because $V(R)$ is sorted, the right values not exceeding a left value a form a prefix. Looping over that prefix and then the rest fixes which operand is larger without comparing them.

7.3 Parallel algorithms on the GPU

A GPU runs thousands of threads with no common order of events, so "keep the first derivation found" has no direct meaning. Two observations restore determinism.

Proposition 7.4 (Derivation by minimum)**.** *Encode a derivation $(L, a, b, \circ)$ of v from S as the integer $\big(\sigma_S(L), a, b, \mathrm{rank}(\circ)\big)$ packed most significant first. Then $\omega(S, v)$ is the minimum of the codes of all derivations, whatever order they are found in.*

Proof. Packing with enough bits per field turns the lexicographic order of Definition 5.2 into integer order, and the minimum of a set does not depend on the order of its elements. □

So threads can fold derivations into a result with an atomic minimum. The second observation is that origins need not be stored at all.

Proposition 7.5 (Sets suffice)**.** *Given the sets $V(L)$ for all $L \subsetneq S$, $\textsc{Find}(S, v)$ computes $\omega(S, v)$ for any $v \in V(S)$. Hence $\tau(S^*, v^*)$ can be recovered from the value sets alone with at most $k^* - 1$ calls to* Find*, one per internal node.*

Proof. The first claim is the second step of the proof of Theorem 7.1, which used only the sets. The tree has k^* leaves and so $k^* - 1$ internal nodes, each of which is the canonical derivation of its own subset and value. □

The CUDA solver uses both. One thread block solves one round in on-chip shared memory. It stores each subset's values in a hash table keyed by subset and value, so duplicates are removed by an atomic compare-and-swap and nothing is sorted. The nodes of the answer tree at the same depth are independent, so they are expanded one level at a time, with all threads searching at once. The PyTorch solver instead keeps every value of every round in one sorted array of keys (size, round, mask, value) and answers membership queries by binary search. It removes duplicates with a stable sort of results generated in canonical order, so the first derivation of each value survives.

Both GPU solvers also avoid building the subsets of size $n - 1$ when the answer uses every number.

Lemma 7.6 (Deferred level)**.** *Let $n \geq 3$. In any split of $[n]$, if one half has size $n - 1$ then the other is a singleton. So $\textsc{Find}([n], t)$ can run with the sets of size $n - 1$ unbuilt: in those splits it scans the singleton and decides each of at most six partners b by a nested $\textsc{Find}(L, b)$ with $|L| = n - 1$, which needs only sets of size at most $n - 2$.*

Proof. The sizes of the two halves add up to n. Nested searches return membership correctly by the second step of Theorem 7.1. □

There are n such splits with at most six partners each, so at most $6n = 36$ nested searches, against 7,140 pairs to build size five. The sets of size $n - 1$ are still built when the whole pool misses the target, because case (b) of Proposition 4.3 needs every proper subset. By the same counting, a subset of size $n - 2$ is a half only in splits of sets of size $n - 1$ or n, where the other half has size 1 or 2. When $n \geq 5$ that other half is the smaller one, so searches scan it and only look up the values of the $(n - 2)$-subset. The CUDA solver therefore keeps those sets as hash tables without a list, listing them by a table scan only in the rare rounds that build size $n - 1$. This cuts its shared memory enough to run three rounds on each multiprocessor at once. For $n = 4$ the argument fails, since the split $2 + 2$ has equal halves, and the solver lists those sets as usual.

7.4 Precomputed and learned answers

Two further versions trade exact text for speed. Their answers are equally good but not always identical.

Lookup table. By Proposition 2.3 the game has only 8,548,200 instances up to the order of the pool, so their answers can all be computed in advance. The table is keyed by the sorted pool, so by Remark 5.9 it stores Solve of a permutation of the dealt pool: the same (d^*, k^*), possibly with different text. Keying on dealt order instead would need about 430 million answers.

Learned proposals with a certificate. A small transformer reads (t, p) and proposes an expression one token at a time, with decoding constrained so that every proposal is legal. A proposal is only a hint, and the solver accepts it on the following certificate.

Proposition 7.7 (Certificate)**.** *Suppose a legal expression e over p has value t and uses k numbers, and* $\textsc{Find}(S, t) = \bot$ *for every S with $|S| < k$. Then $d^* = 0$, $k^* = k$ and e is equally good as* $\mathrm{Solve}(t, p)$.

Proof. The expression shows that $t \in V(\mathrm{lv}(e))$, so $d^* = 0$ and $k^* \le k$, and the failed searches show $k^* \ge k$. □

Checking the certificate needs only subsets of size up to $k - 2$ built, so an accepted proposal saves building size $k - 1$. A rejected or missing proposal falls back to the full algorithm, so the hybrid is never worse than the solver. In practice the model proposes a shortest answer too rarely, on 7% of six-number rounds, for the saving to outweigh the cost of running it.

8 Empirical facts

The benchmark set `bench/data/rounds10k.txt` holds 10,000 game-distributed rounds. Its reference answers `ref10k.tsv` are $\mathrm{Solve}(t, p)$ as computed by the Rust solver.

Outcome	Rounds
Exact ($d^* = 0$)	9,776
using 1 number	3
using 2 numbers	81
using 3 numbers	860
using 4 numbers	3,622
using 5 numbers	4,063
using 6 numbers	1,147
Closest at $d^* = 1$	207
Closest at $d^* = 2$	13
Closest at $d^* = 3$	2
Closest at $d^* = 4$	2

Table 2: Optimal distance and count over the 10,000 benchmark rounds.

Two consequences shape the solvers. First, almost every round is exact, so by Proposition 6.2 the full pool's value set is almost never needed. The five-number sets are needed only by the 11% of rounds whose answer uses all six numbers, and the GPU solvers avoid even those by Lemma 7.6, building them only for the 2% of rounds that miss. Second, the work is dominated by the largest subsets. The reachable sets grow roughly eightfold with each extra number, and most of the cost lies in combining two halves of a five- or six-number subset.

9 Performance

Every implementation was timed on the same 10,000 benchmark rounds. Table 4 lists the results, copied from the repository's `PERFORMANCE.md`, fastest first.

$\lvert S\rvert$	mean $\lvert V(S)\rvert$	largest $\lvert V(S)\rvert$
1	1.0	1
2	3.1	4
3	17.1	28
4	133.8	228
5	1,079.1	2,181
6	8,154.1	18,946

Table 3: Sizes of reachable sets over the first 300 benchmark rounds. The largest value seen was 25,200,000.

Method. The machine is an Intel Core i9-13900HX with an NVIDIA RTX 4070 Laptop GPU, running Linux 7.0. CPU benchmarks are pinned to one performance core. *Solve time* is the best of three runs, measured inside a harness for each language, so that start-up and output are excluded. Python and bwBASIC are timed on a 1,000-round set because the full set takes over a minute. The just-in-time compiled languages are timed after three untimed warm-up passes. The GPU harnesses warm up first too, so that CUDA start-up, kernel compilation and the GPU's climb from its idle clock are excluded. *Straight port* is the solve time of each language's first faithful translation, before any language-specific optimisation. *Whole program* is the mean wall-clock time of each language's own executable dealing and solving one random round, over 300 runs (10 for the two PyTorch versions), so it includes process and runtime start-up.

The twelve newest languages (FreeBASIC, BASIC, WebAssembly, F#, Haskell, Kotlin, Common Lisp, OCaml, Ruby, Julia, Erlang and Elixir) were measured later on 2026-10-09, in sessions in which C measured 0.0182–0.0185 ms, and ISPC on 2026-10-10, when C measured 0.0183 ms. The four GPU and precomputed rows were measured a day after the original set, in a quieter session in which C measured 0.0184–0.0188 ms rather than 0.0208 ms. Comparisons between those rows and the others therefore carry roughly 10% uncertainty. The GPU rows are throughput, with all 10,000 rounds submitted as one batch, not the latency of a single round.

Observations.

(1) *The algorithm matters more than the language.* The compiled languages from Zig to Java lie within a factor of two of one another, while every one of them gained between two and twenty-one times from the techniques of Sections 7.1 and 7.2. The largest single gain, Rust's 174×, came from moving its first version to the staged search with early exit (Proposition 6.2) and searching rather than building (Lemma 6.3), together with a compact bitset store.

(2) *Compilers set the floor.* C, C++, Zig, FreeBASIC and hand-written assembly finish within 8% of one another, because they share one design and differ only in code generation. Garbage-collected and managed languages come close when written over flat buffers, and interpreters trail C by between nine times and, for the line-numbered bwBASIC, about fifteen thousand times. WebAssembly in Node comes within 25% of native C. The one solver below that floor, ISPC, changes the design to suit SIMD lanes (Section 10.3).

(3) *Parallelism needs batches.* The CUDA solver is about 40 times faster than C per round, but only when thousands of rounds are in flight, one per thread block. The PyTorch solver takes 11 ms for a single round, slower than plain Python, and only passes C at around 10,000 rounds per batch. Their whole-program times, 0.2 and 3.2 seconds, are almost entirely runtime and GPU start-up. The parallel formulation of Section 7.3 makes the GPU versions exact, but it does not make them suitable for one round at a time. Sharing one round across CPU threads gains at most a factor of two (Section 11).

Implementation	Solve ms/round	Straight port ms/round	Gain	Whole program ms/run	Answers
Rust + CUDA (GPU, batched)	0.000442	0.0107[a]	24×	199	identical
Lookup table (Python)	0.00069	–	–	13.4	31% ties
PyTorch on the GPU (batched)	0.0157	0.142[a]	9.0×	3,218	identical
ISPC (AVX2, four 64-bit lanes)[d]	0.0115	0.078	6.8×	0.35	identical
Zig (ReleaseFast)	0.0203	0.083	4.1×	0.48	identical
C (Clang)	0.0208	0.118	5.7×	0.39	identical
Julia 1.13	0.0208	0.259	12.5×	548	identical
C++ (Clang)	0.0213	0.086	4.0×	0.73	identical
FreeBASIC (GCC back end)	0.0215	0.123	5.7×	0.50	identical
x86-64 assembly	0.0218	0.043[a]	2.0×	0.23	identical
WebAssembly (C in Node)	0.0229	0.243	10.6×	17.4	identical
Fortran (gfortran)	0.0247	0.109	4.4×	0.52	identical
Ada (GNAT)	0.0260	0.260	10×	0.78	identical
C# (.NET 10)	0.0274	0.212	7.7×	21.9	identical
Swift	0.0274	0.212	7.7×	1.52	identical
F# (.NET 10)	0.0287	0.332	11.6×	30.3	identical
Go	0.0330	0.190	5.8×	0.64	identical
Pascal (Free Pascal)	0.0352	0.752	21×	0.38	identical
PyTorch with transformer proposals	0.0354	–	–	3,223	8% ties
Haskell (GHC)	0.0360	0.639	17.8×	0.82	identical
Kotlin (JVM)	0.0373	0.149	4.0×	48.2	identical
Java (OpenJDK 25)	0.0391	0.154[b]	3.9×	34.2	identical
Rust	0.0451	9.7[a]	215×	0.51	identical
Common Lisp (SBCL)	0.0472	0.446	9.4×	1.51	identical
OCaml	0.0537	0.450	8.4×	0.64	identical
Erlang (BEAM)[c]	0.0575	0.494	8.6×	73.7	identical
Elixir (BEAM)[c]	0.0576	0.729	12.7×	83.6	identical
JavaScript (Node 24)	0.0848	0.381	4.5×	24.3	identical
COBOL (GnuCOBOL)	0.173	2.07	12×	1.30	identical
LuaJIT	0.186	0.61	3.3×	2.11	identical
Ruby with RJIT	0.291	–	–	29.0	identical
Forth (`gforth-fast`)	0.318	0.864	2.7×	4.28	identical
Forth (`gforth`)	0.399	1.036	2.6×	7.02	identical
Ruby 3.3	0.494	2.22	4.5×	29.6	identical
Lua 5.4	0.550	1.10	2.0×	2.44	identical
Python	7.23	12.0[a]	1.7×	17.6	1.5% ties
BASIC (bwBASIC, line-numbered)	306	1,114	3.6×	282	identical

Table 4: Solve time per round over the 10,000 benchmark rounds (1,000 for Python and bwBASIC), the time before optimisation, and the time for a whole program run. [a]First version rather than a straight port of another language. [b]Re-measured with the current harness (Section 10.3); 0.257 ms was recorded originally. [c]Defers the four- and five-number subsets, extending Lemma 7.6, and so does less work than the other CPU versions; Erlang built like the others takes about 0.22 ms. [d]Measured on 2026-10-10 when C took 0.0183 ms, so 1.6 times faster than C (Section 10.3). "Ties" are equally good answers that differ in text from Solve; no implementation gives a worse answer.

(4) *Precomputation wins on latency.* Because the instance space is finite (Proposition 2.3), a 248 MB table answers in 0.7 µs from plain Python, faster than any solver except CUDA. The price is that its text matches Solve for only 69% of rounds (Remark 5.9).

(5) *Learning did not pay.* The transformer's certified proposals (Proposition 7.7) never give a wrong answer, but they too rarely save the expensive levels to cover the cost of running the model, so the hybrid is 2.3 times slower than the PyTorch solver it guides.

10 Straight ports against optimised versions

Every language was implemented twice. The *straight port* is a faithful translation of Algorithm 1 as the Rust reference writes it. It uses the language's ordinary data structures, the library sort, a branching binary search and the `%` operator for the division test. The *optimised version* keeps the same algorithm and the same output, and changes only how the work is done. Both versions of every language were checked against the reference answers. Table 5 summarises what changed, and the rest of this section explains why the changes help.

10.1 Changes shared across languages

Most optimised versions adopt some or all of the refinements of Section 7.2. In rough order of impact they are:

(a) a 16-byte entry holding the value, operator, masks and both operands, written without a branch and kept only if the bitset shows the value is new;

(b) the reciprocal division test of Lemma 7.3 in place of an integer division;

(c) each pair loop split at the left value, so the larger operand is known without a comparison;

(d) sorting by rank in the bitset (Lemma 7.2), which needs no comparisons;

(e) branchless binary search and other unpredictable comparisons turned into conditional moves or arithmetic masks;

(f) the bitset sized by the bound $M(S)$, so it grows only when larger subsets are built.

Each of these is an exact transformation of Build or Find, so none can change Solve. The straight ports already had the staged search, the early exit and the partner lookups, because the Rust reference they copy has them.

10.2 What the gains reveal

The changes fall into a handful of recurring kinds, and which kinds matter says more about each language's implementation than about the language itself.

Memory management. Several of the largest single gains had nothing to do with arithmetic. Zig's default allocator and Free Pascal's heap manager both returned large blocks to the kernel on every free. Since each solve allocates its own memory, Zig's version took 255,000 page faults over a run. Switching to the C library's allocator halved both run times. Go spent a third of its time in the garbage collector rescanning the harness's 20,000 small objects, and gfortran zeroed whole arrays because of default component values. In each case the straight port was paying for the runtime rather than the algorithm.

Language	Straight ms/round	Optimised ms/round	Gain	Principal changes
ISPC	0.078	0.0115	6.8×	values only, deduplicated by bit OR; pair loops across four 64-bit lanes; lists read from bitsets; bit-test partner checks
Zig	0.083	0.0203	4.1×	C allocator instead of the default, which returned memory to the kernel on every free; buffers grown by doubling; branch hint for a conditional move in the search
C	0.118	0.0208	5.7×	all of (a)–(f); Clang instead of GCC; binary search adds `(value <= target) * half`; subsets listed in size order up front
Julia	0.259	0.0208	12.5×	values listed straight from the bitset with `trailing_zeros`, so no sort; an `llvmcall` select marked `!unpredictable` to force conditional moves
C++	0.086	0.0213	4.0×	the C design with templates and lambdas; buffers that skip zero-filling; Clang instead of GCC
FreeBASIC	0.123	0.0215	5.7×	port of the optimised C with typed pointers and macros; `-march=native` for a one-instruction popcount; `IIf` replaced by a mask
Assembly	0.043[a]	0.0218	2.0×	rank sort; branchless stores; conditional moves in the search; branchless division test
WebAssembly	0.243	0.0229	10.6×	the optimised C compiled unchanged with portable stand-ins for three intrinsics; quotient rounded by adding $1.5 \cdot 2^{52}$
Fortran	0.109	0.0247	4.4×	`-O3` so the store is inlined; no default component values, which zeroed whole arrays; arrays passed separately so they cannot alias; masks instead of `merge`
Ada	0.260	0.0260	10×	runtime checks suppressed with `-gnatp`; cross-unit inlining; popcount and rounding intrinsics; modular type for masks
C#	0.212	0.0274	7.7×	near line-for-line port of the optimised C using `unsafe` pointers, native memory and `PopCount`
Swift	0.212	0.0274	7.7×	unsafe pointer buffers; wrapping arithmetic; `-Ounchecked`; ternaries replaced by arithmetic masks
F#	0.332	0.0287	11.6×	port of the optimised C# over native memory; conversions through signed `int64`; arithmetic mask in the division test
Go	0.190	0.0330	5.8×	flat harness data so the garbage collector stops rescanning; `GOAMD64=v3`; masks instead of branches; lookup trimmed to fit the inliner
Pascal	0.752	0.0352	21×	C allocator via `cmem`; 64-bit floating constants so the reciprocal is exact
Java	0.154[b]	0.0391	3.9×	entries packed into a `long[]`; `Long.bitCount` rank sort; straight-line partner checks
Kotlin	0.149	0.0373	4.0×	port of the optimised Java with `LongArray` entries
Haskell	0.639	0.0360	17.8×	`ST` over unboxed `PrimArrays`; bitset with rank sort; values only, origins rebuilt for the answer
Common Lisp	0.446	0.0472	9.4×	port of the optimised C with typed arrays, `logcount` and `(safety 0)`
OCaml	0.450	0.0537	8.4×	byte bitset with radix sort; integer-only `max` and `min`; `Bigarrays` to spare the collector; values only
Rust	9.7[a]	0.0451	215×	staged search with early exit; values only while searching; compact entries with a bitset grown by subset size; partner lookups; none of (a)–(f) yet
Erlang[c]	0.494	0.0575	8.6×	rewritten to Elixir's design, with tuples for binary search and subset tables as constants
Elixir[c]	0.729	0.0576	12.7×	values only with `:lists.usort`; four- and five-number subsets deferred; ordered partner checks; outward search for closest rounds
JavaScript	0.381	0.0848	4.5×	four 32-bit words per entry in a `Uint32Array`; 32-bit bitset words with a hand-written popcount; no typed-array views in the insertion sort
COBOL	2.07	0.173	12×	hot operations kept out of GMP decimal arithmetic: shift-and-add multiplication, long division, byte stamps and a byte radix sort
LuaJIT	0.61	0.186	3.3×	each origin packed into one number; a radix sort written in Lua; hash tables pre-sized
Ruby	2.22	0.494	4.5×	a Hash per subset with `key?` checks; split pair loops; whole-pool pass unrolled
Forth (fast)	0.864	0.318	2.7×	byte stamp per value; byte radix sort; pointer stepping; hot words compiled in place
Forth	1.036	0.399	2.6×	as above
Lua 5.4	1.10	0.550	2.0×	each origin packed into one number; `table.sort` kept
Python	12.0[a]	7.23	1.7×	values without origins while searching; origins rebuilt only for the answer
bwBASIC	1,114	306	3.6×	one hash table of values without origins; hot subroutines placed first because `GOTO` searches from the top
PyTorch	0.142[a]	0.0157	9.0×	expressions typeset on the GPU; pair generation fused by `torch.compile`; shorter half scanned; five-number subsets deferred; illegal results dropped before sorting
Rust + CUDA	0.0107[a]	0.000442	24×	split tables computed once; phases flattened across threads; four-number subsets as tables only; answer unfolded a level at a time

Table 5: Straight port against optimised version for each language. [a]First version rather than a straight port. [b]Re-measured (Section 10.3). [c]Defers the four- and five-number subsets, extending Lemma 7.6, and so does less work than the other CPU versions.

Branches. The binary search in FIND and the duplicate test in BUILD take each direction about half the time, so a branch there mispredicts constantly. Every compiled language gained by making them branchless, but each needed a different way to ask for it. Zig has an explicit hint. Clang ignored its hint, so the C version multiplies by a comparison. Go has no way to ask, so its version uses arithmetic masks, and so do Fortran and Swift, whose compilers turned `merge` and the ternary operator back into branches.

Slow primitives. Integer division is slow, so the reciprocal test of Lemma 7.3 pays off in every compiled language. Free Pascal first stored the constant 1.0 in single precision, which made the reciprocal too inexact for the lemma to hold until the precision was raised. COBOL is the extreme case. GnuCOBOL compiles multiplication, division and bitwise operations into calls to GMP's decimal arithmetic, which took 65% of the time. The optimised version restricts its hot paths to the operations that compile to native code and builds the rest from them, which cut decimal arithmetic to 3%.

Sorting. Library sorts compare, and comparisons through a function pointer are slow. This is why straight C, which uses `qsort`, started slower than straight C++, whose `std::sort` is inlined for the entry type. The rank sort of Lemma 7.2 removes comparisons entirely where a population count is cheap. COBOL and Forth use a byte radix sort instead, and so does LuaJIT, which cannot compile `table.sort`. Lua 5.4 kept `table.sort`, because its interpreter runs a radix sort written in Lua three times slower than the built-in.

Safety checks. Ada's straight port keeps the language's default bounds and overflow checks, and its optimised build suppresses them alongside inlining and native code generation. Swift's `-Ounchecked` took it from 0.043 to 0.030 ms. Removing checks is only safe once the code is known to be in range, so the optimised Ada solver was also run over all 10,000 rounds with checks enabled, with identical answers.

Interpreters. In Lua and Forth the cost tracks the number of operations the interpreter dispatches, not the machine work behind them. The gains there come from doing less per value: packing an origin into one number instead of a table, stamping a byte with the subset's mask instead of clearing a bitset, and stepping pointers rather than computing indices.

Line-numbered BASIC. bwBASIC interprets the program text almost directly. It costs about 2.4 µs per statement, plus about 1 µs per operator, and a fifth of its time is spent in `strcmp`. It finds every `GOTO` and `GOSUB` target by searching from the start of the program. A jump to a line near the end therefore costs about 3 µs, against 0.4 µs near the start, and the optimised solver puts its hot subroutines first, so that line numbers themselves become a performance decision. Its 3.6-fold gain otherwise came from Proposition 7.5: it keeps one hash table of values, with no sorting and no stored origins, and rebuilds the few origins the answer needs by search.

Python. The Python version took a different route, using Proposition 7.5. It stores only the value sets while searching and rebuilds origins only for the nodes of the final answer. That saves about 40%. The interpreter's cost per value still dominates, so Python remains 350 times slower than C.

What did not carry over. A SIMD rewrite of the assembly was ruled out because removing duplicates is serial: whether a value is new depends on every value before it. Profile-guided optimisation gave at most 3% in C, Go and Fortran. Radix sort lost to rank sort in C at every

sparsity tested. The GPU versions are a different story. Their gains come from restructuring for parallelism (Section 7.3) rather than from any of (a)–(f).

10.3 Surprises examined

Adding Haskell, OCaml, Common Lisp, F#, Kotlin, Ruby, WebAssembly, Erlang, Elixir and ISPC produced several results that go against the usual reputations of these languages. This subsection gives the evidence for each one and our reading of the cause. Times are best of three, pinned to one core. Hardware counters come from `perf stat`. To leave out start-up and input parsing, they difference a run of six passes over the 10,000 rounds against a run of one pass. Where we infer a cause rather than measure it, we say so.

Language	ms/round	instructions	cycles	IPC	branch misses	loads
C (Clang)	0.0187	324,000	93,400	3.47	797	52,300
Go	0.0303	541,000	154,700	3.50	985	106,300
Haskell (GHC)	0.0366	596,000	186,500	3.20	1,576	140,800
Common Lisp (SBCL)	0.0487	800,000	237,500	3.37	845	168,800
Rust	0.0503	608,000	259,900	2.34	3,665	134,400
OCaml	0.0541	1,102,000	272,500	4.04	1,676	203,400

Table 6: Hardware counters per round for the ahead-of-time compiled solvers, on the 10,000 benchmark rounds, with the Rust solver as it was before the bitset fix described in Section 13.

Common Lisp nearly matches Rust. SBCL is a dynamically typed Lisp, yet when Table 6 was measured its optimised solver, at 0.049 ms, beat the Rust solver, at 0.050 ms. The counters show why. Rust executes *fewer* instructions than Lisp, but it suffers 3,665 branch mispredictions per round against Lisp's 845, which is close to C's 797. At roughly 15–20 cycles each, the extra 2,800 mispredictions cost about 50,000 cycles. That is most of the difference, and it drags Rust's instructions per cycle (IPC) down to 2.34. The Lisp version is a direct port of the optimised C design, including the branchless append and binary search of refinements (a) and (e). The Rust solver has not yet had any of (a)–(f). So this result is about design rather than language: SBCL with full type declarations and `(safety 0)` compiles to plain machine arithmetic, and the branchless design matters more than the better compiler. A later fix stopped Rust zeroing a bitset sized for the whole pool on every solve (Section 13). That brought Rust to 0.045 ms against Lisp's 0.047, but left it at about 3,800 branch mispredictions per round. Profiling also shows that 11–15% of the Lisp solver's cycles are in `memset`, called from SBCL's allocator (`lisp_alloc`). The allocator zeroes each fresh region of memory it hands out, and the solver allocates its arrays anew for every solve. Lisp therefore has further headroom.

Haskell comes within a factor of two of C. A lazy, purely functional language might be expected to trail far behind, but the optimised Haskell solver runs at 0.037 ms. That puts it between Go and Java, and ahead of OCaml, Lisp and Rust. Two-thirds of its cycles are in a single function, `$wsolve`. This is GHC's *worker*: after strictness analysis has shown that every argument will be evaluated, GHC splits off a version that takes them unboxed, in registers. The state lives in unboxed `PrimArray`s inside the `ST` monad, which compile to plain loads and stores, and the garbage collector's share is under 5%. What remains is a code-generation gap. Haskell executes 1.84 times C's instructions and suffers twice C's branch mispredictions. We think, without having checked the assembly, that GHC's native code generator emits real branches where Clang emits conditional moves.

OCaml trails Haskell. OCaml is usually the faster of the two, but here it is the slowest of the compiled functional languages. Its counters are unusual. It runs at 4.04 instructions per

cycle, the best of any solver, so the processor is not stalling. It simply executes 3.4 times as many instructions as C. We attribute this to three causes. First, OCaml integers are 63 bits with a tag bit, so most arithmetic needs extra instructions to adjust the tag. Second, OCaml has no population-count primitive, so the rank sort of Lemma 7.2 is unavailable: the solver keeps a byte per 8 values and sorts with a multi-pass radix sort instead. Third, the compiler installed here lacks the *flambda* optimiser, so it inlines less. Garbage collection costs about 3% after the large arrays were moved to `Bigarray`s. Haskell wins because GHC can work on raw unboxed integers, which suits an integer-array workload like this one.

WebAssembly is 3.3 times faster than JavaScript in the same engine. Both run in V8, yet the C solver compiled to WebAssembly takes 0.023 ms, against 0.076 ms for the hand-optimised JavaScript solver. That is only 1.24 times native C. The difference is in what each format lets the engine assume. JavaScript's bitwise operators work on 32 bits, so the JavaScript bitset has twice as many words and counts bits with shifts and masks. Every number must also be checked for its type at run time. WebAssembly has native 64-bit integers and a `popcnt` instruction, its types are fixed in advance, and Clang has already optimised the code before V8 sees it. On 64-bit hosts V8 also avoids most bounds checks on WebAssembly memory accesses by reserving guard pages around them. The remaining 24% over native comes from about 1.5 times native's instruction count. Neither BMI2 nor the direct rounding conversion used by the C version is available, and V8's compiler optimises less aggressively than Clang. The straight port suffers far more, running 2.1 times slower than its native build. About half of its time goes to the C library's `qsort`, which calls the comparison function through an indirect call. Another fifth goes to V8's out-of-line handling of the variable-length `memory.copy` instructions behind `realloc`. The optimised solver uses neither sorting by comparison nor frequent reallocation, which is why WebAssembly costs it so little.

F# needed one change C# did not. F# and C# share the .NET runtime and its compiler, and the optimised F# solver, a port of the C# design, ends within 10% of it. The largest single gain, from 0.032 to 0.0275 ms, came from converting between integers and floating point through *signed* 64-bit integers. Before AVX-512, x86-64 has no instruction that converts an unsigned 64-bit integer to or from a double, so the compiler emits a branching sequence of several instructions for each conversion. The values stay below 2^{27} (Proposition 3.5), so the signed conversion is exact. The straight ports differ more: F#'s takes 0.33 ms and C#'s 0.19 ms. A profile of the F# port puts 15% of its cycles in runtime type tests (`IsInstance`) and in F#'s generic structural comparison. Idiomatic F# compares tuples structurally, for instance in `Seq.minBy` with a tuple key, and that comparison goes through a generic comparer that discovers types at run time. F#'s `option` type also allocates an object for every `Some`, where C# uses a null reference.

A surprise that was not. At first the Kotlin straight port, at 0.15 ms, appeared much faster than Java's recorded 0.257 ms. We suspected Kotlin's `inline` functions, which remove the per-pair lambda calls Java's port makes. Re-timing Java's straight port with the current harness gives 0.154 ms, the same as Kotlin. The old figure dates from before the harness ran three full warm-up passes, so it measured HotSpot part-way through compiling. Table 4 and Table 5 now show the new figure. The optimised Kotlin and Java solvers also tie, at 0.0374 and 0.0376 ms, as expected of two languages compiling to near-identical bytecode.

Interpreted Ruby beats interpreted Lua. The optimised Ruby solver runs at 0.51 ms in the standard interpreter, slightly ahead of Lua 5.4's 0.55 ms. It runs at 0.31 ms under Ruby's RJIT compiler. Lua is normally considered the lighter and faster interpreter. The profiles point to sorting. In Lua 5.4, 21% of all cycles are in `lua_geti`, `lua_seti` and `lua_compare`, the general-purpose C interface that `table.sort` uses to read, write and compare every element.

Ruby's `Array#sort` on integers instead calls the C library's merge sort with a fast path for comparing small integers, which takes 4% of its cycles. Both interpreters spend most of their remaining time in hash-table lookups, where they are evenly matched.

Two languages on one virtual machine. Erlang and Elixir compile to the same BEAM bytecode and run on the same just-in-time compiler, yet their first optimised solvers differed by a factor of 2.8: 0.162 ms for Erlang against 0.058 ms for Elixir. The difference was entirely in design. Building a value costs 50–80 ns on BEAM, against a few nanoseconds in C, so it pays to build less even at the price of more searches. Elixir left both the four- and five-number subsets unbuilt, extending Lemma 7.6 by one level, stopped at the first matching partner, and searched outward from the target ($t \pm 1$, $t \pm 2$, up to 8) before building the whole pool in the 2% of rounds that miss. Those rounds had taken 38% of its time. Rewritten to the same design, Erlang runs at 0.0575 ms, level with Elixir. Removing each idea from the rewritten version in turn shows that the extra deferred level gives the largest gain and the outward search about a quarter. Building none of the subsets was the opposite of what PyTorch found, where deferring the four-number subsets as well cost 30%.

SIMD lanes break the C floor. ISPC compiles a C-like program to run on several SIMD lanes in lockstep, like a GPU thread group inside one core. Its solver takes 0.0115 ms per round against C's 0.0183 ms in the same session, 1.6 times faster, the only CPU solver to beat C. It executes 163,000 instructions per round against C's 324,000, with 358 branch mispredictions against 794. The lanes cannot simply be pointed at Algorithm 1, because the first derivation of each value must win. The solver therefore stores values only, so that removing duplicates is an order-free bit OR, and finds each origin again for the answer, which is exact by Proposition 7.5. The longer half of each pair loop is spread across the lanes, which compute all four results, their legality and the reciprocal division test (Lemma 7.3) together. Sorted lists are read straight out of each subset's bitset, partner checks become single bit tests, and the sets of sizes $n - 1$ and $n - 2$ are built but never listed, since by the counting after Lemma 7.6 searches only look them up. A scalar C version of the same structure runs at 83,000 cycles per round against 62,000 for ISPC and about 95,000 for the C solver, so about a quarter of the gain comes from the lanes and an eighth from a structure C could adopt. The best gang was the *narrowest*: with four 64-bit lanes, eight 32-bit lanes needed 7% more cycles and sixteen lanes 32–41% more. Setting a bit is a read-modify-write for each lane slot, so idle lanes cost as much as busy ones. C's trick of sending illegal results to the value 0 also turned into a chain of dependent updates to one word under SIMD; giving each lane its own spare word saved 19% of ISPC's cycles but only 1–2% of C's.

What the surprises have in common. Each one has the same explanation underneath: what matters is how many instructions the hot loops execute and how often they mispredict a branch, not the language's general reputation. A language does well when it can express the design of Section 7.2 directly. That means unboxed integer arrays, a population count, and branchless selection. It does badly when its runtime inserts work into those loops: tag bits, type tests, generic comparison, allocation zeroing or indirect calls.

11 Sharing one round across threads

The GPU solvers of Section 7.3 are fast only on batches. Seven of the CPU solvers (C, Rust, Go, Java, Julia, Erlang and Elixir) also have a separate version that splits a *single* round across threads. They use three kinds of independence in Algorithm 1, chosen so that the answer does not depend on which thread does what.

(i) *Builds within a level.* The value set of S depends only on the sets of its proper subsets, so all subsets of one size can be built at once, each by a single thread in the usual order. Removing duplicates inside one subset stays serial, because "first derivation found" needs a fixed order.

(ii) *Searches across splits.* The splits of $\Sigma(S)$ are searched by several threads, and the earliest hit in split order wins. A shared atomic records the best position found so far, and threads skip splits that come after it.

(iii) *The closest pass.* Each thread finds the minimum canonical key κ over its share of the splits, and the minima are combined, as in the derivation-by-minimum argument used by the GPU solvers.

Creating threads inside a solve costs more than C's entire solve, so each version keeps a worker pool alive between solves, holding no round data. Rounds that finish within four numbers stay on one thread.

Solver	1	2	4	8	16 (P+HT)	16 (P+E)	32	Best
C	0.0187	0.0159	0.0142	**0.0132**	0.0153	0.0148	0.0236	1.42×
Julia	0.0208	0.0173	**0.0154**	0.0161	0.0181	0.0180	0.0233	1.35×
Rust	0.0461	0.0326	0.0253	**0.0228**	0.0258	0.0290	0.0416	2.02×
Java	0.0334	0.0275	**0.0222**	0.0266	0.0342	0.0421	0.0465	1.50×
Go	0.0296	0.0272	**0.0267**	0.0317	0.0429	0.0544	0.0660	1.11×
Elixir[c]	0.0549	0.0416	**0.0316**	0.0318	0.0469	0.0440	0.0595	1.74×
Erlang[c]	0.0568	0.0444	**0.0330**	0.0355	0.0539	0.0453	0.0761	1.72×

Table 7: Milliseconds per round for the parallel solvers by number of threads, best of three passes over the 10,000 benchmark rounds, with every answer identical to Solve. Up to 8 threads, each thread has its own performance core. At 16, P+HT uses both hardware threads of the eight performance cores and P+E adds eight efficiency cores. "Best" is the largest speed-up over the same code on one thread. [c]As in Table 4.

Table 7 shows the result on the 32 logical CPUs of the test machine. Its findings are as follows.

The gain is small and stops at four to eight threads. The best speed-ups are between 1.1 and 2.0, and every solver is at its fastest with 4 or 8 threads. The reason is Amdahl's law applied to Table 3's shape. About 46% of rounds are settled by the four-number subsets and never go parallel. At the five-number level there are only six subsets, so (i) can keep at most six threads busy. Within each subset, removing duplicates is serial, and in Rust it alone was 37% of the time. Rounds that use all six numbers gain the most, 1.7–2.9 times at four threads, and so do the rounds that miss, which run the closest pass (iii).

More threads make it slower. At 32 threads only Rust still beats its single thread. C is 1.26 times slower than on one thread, Erlang 1.34, Java 1.39 and Go 2.2. Each phase ends at a barrier and so waits for its slowest thread, which past 8 is either an efficiency core, running at roughly half the speed, or a hardware thread sharing its core with another worker. Every extra thread also adds a hand-off, scratch memory to allocate for every solve (the no-reuse rule costs C 44% more build work) and pressure on the shared cache, while having no work to take on. Hardware threads and efficiency cores did about equally badly at 16 for C, Julia and Elixir. The garbage-collected Go and Java did worse with efficiency cores, and Erlang better; we did not measure why.

Only spinning pools pay. A round lasts 20–60 µs, and a thread that has gone to sleep takes several microseconds to wake. Handing work to a hand-written pool whose idle workers spin cost 0.3 µs in Rust and 0.6 µs in Java, against 8.5 µs for Rust's `rayon` and 11–12 µs for Java's fork-join pool and virtual threads. With fork-join, the parallel solver was slower than the serial one. Go's goroutines also lost to a spinning pool. Julia's `@spawn` came closest, 7% behind.

The memory manager decides the managed languages. At Go's default collector setting, four workers were slower than one, because each allocates a bitset of up to about 1 MB per round and the collector ran far more often; the pool now raises its threshold. Julia's collector stops every thread, so its per-thread bitsets are allocated outside it, with `calloc`. On BEAM, processes share no memory. The first parallel Erlang copied every built value list to each worker and gained only 1.4 times. Like Elixir, it now has each worker rebuild the small subsets itself from the six numbers and the target, pass the three-number subsets once as a shared binary, and claim tasks through atomic counters. With that design the two BEAM languages match again, at 1.72 and 1.74 times. Both also needed scheduler settings: idle BEAM schedulers sleep, and Elixir's spare "dirty" schedulers spun and took processors from the real work, so 2 threads were slower than 1 until they were turned off.

A race that only 32 threads exposed. The first sweep never finished Elixir at 32 threads: the virtual machine spun on every CPU for three hours. The caller read the best position of (ii) once and then waited for each split before it to be marked done. Meanwhile one worker could lower the best, and another would then skip a split at or after the new best without marking it, correctly, since it can no longer matter. The caller kept waiting for that split against the stale best. Re-reading the best on every poll and stopping on reaching it removes the race. The answer cannot change, because the winning split's result is stored before the best is lowered. The window opens only when about 30 workers really run at once, which is why 4 or 8 threads never showed it. The Erlang port, which copied the design, hit the same deadlock about once in 15 runs with four schedulers before it received the same fix.

Summary. Parallelism inside one round of a single game gives at most a factor of two, and only by careful engineering. A batch is a different matter: the CUDA solver of Section 7.3 is about forty times faster than one core, because there each round is independent and needs no barriers.

12 How often is there no solution?

Call an instance *solvable* when its target is reachable, that is when $d^* = 0$. The instance space is finite (Proposition 2.3), so the share of unsolvable rounds is a definite number, and it can be found by solving every instance.

Theorem 12.1 (Unsolvable rounds)**.** *Exactly* 380,523 *of the* 8,548,200 *instances are unsolvable, and* 1,217 *of the* 9,498 *pools reach every target. Under the game's distribution*

$$\Pr[\textit{no exact solution}] = \frac{16{,}970{,}693}{837{,}216{,}000} \approx 0.020270,$$

about one round in 49. Given one large number it is $627{,}787/27{,}907{,}200 \approx 0.02250$*, and given two it is* $236{,}059/13{,}081{,}500 \approx 0.01805$.

Proof. By Theorem 3.4 the reachable values of a pool are $\bigcup_S V(S)$, and each $V(S)$ is determined by the splits of S. A checker written for this purpose, sharing no code with any solver, evaluates that recursion literally for all 63 non-empty subsets of each pool in 64-bit arithmetic with overflow checks, and lists the targets outside the union. The C solver of Section 7.2 was run

on all 8,548,200 instances, and the GPU-built lookup table of Section 7.4 holds an answer for each. The three sources agree on every instance, both on whether it is solvable and on the best distance d^*. Summing the deal probability of each pool, $\frac{1}{2}\prod_i \binom{2}{c_i}/\binom{20}{6-\ell}\cdot 1/\binom{4}{\ell}$ for small numbers with multiplicities c_i, times its share of missed targets, gives the fractions, computed in exact rational arithmetic. The weights sum to 1. □

The two halves of the theorem rest on different kinds of evidence. A solvable instance has a short *certificate*, an expression that anyone can evaluate, and the checker writes one for each of the 8,167,677 solvable instances. A separate 100-line evaluator accepts all of them, and also accepts the 8,167,677 exact answers in the lookup table, a second independent set. An unsolvable instance has no such certificate in general: showing that *no* expression reaches the target needs the whole search. That half rests on two programs of different design agreeing, which Section 14 strengthens with machine-checked proofs.

Counted over distinct instances rather than dealt rounds, the share is 380,523/8,548,200 ≈ 4.45%, more than twice as high. The hard pools are those with repeated small numbers, and the deck makes them unlikely: a pool with two 1s misses 12.7% of targets and none reaches every target. As a check on both the dealer and the weights, a million rounds dealt by the Python program missed 20,365 times, a 95% interval of $[0.02009, 0.02064]$ around the exact 0.02027.

How far off. When there is no exact answer it is almost always close. The best distance is 1 in 91.8% of unsolvable rounds, its mean is 20,501,954/16,970,693 ≈ 1.208, and the probability of finishing more than 10 away is about 4×10^{-5}. The worst case is $d^* = 324$, for target 999 from the pool $1, 1, 2, 2, 3, 25$, whose largest value is 675.

Holes, not range. Unsolvable rounds are almost never out of reach.

Proposition 12.2 (Largest value)**.** *The largest reachable value of a pool is $B([n])$, where $B(\{i\}) = p_i$ and $B(S)$ is the maximum over the splits $\{L, R\}$ of S of* $\max(B(L)B(R), B(L) + B(R))$.

Proof. Every legal result of $x \geq y \geq 1$ is at most $\max(xy, x + y)$, so induction on $|S|$ bounds every value by B. Conversely, with $x \geq y$, if $y = 1$ the larger option $x + 1$ is made by addition, and if $y \geq 2$ then $xy \geq x + y$ and multiplication is legal, so a tree attaining B is legal. □

Only three game pools have a largest value below 999: $1, 1, 2, 2, 3, 25$ (675), $1, 1, 2, 2, 4, 25$ and $1, 1, 2, 3, 3, 25$ (both 900). They account for 522 unsolvable instances. The other 380,001 are *holes*, targets below the pool's largest value that no expression hits. The cruder bound of Proposition 3.5 is at least 3,743 for every game pool and rules out none of them.

Hard and easy targets. Exactly 14 targets are reachable from every pool: 100, 101, 102, 104, 105, 106, 111, 114, 119, 125, 150, 200, 300 and 450. The miss rate rises steeply with the target, from 0.04% in the 100s to 5.6% in the 900s, and the hardest targets are 967, 937, 941, 933 and 947, each missed by about 11.5% of dealt rounds. Four of those five are prime, and primes are harder in general.

Proposition 12.3. *If a prime t is reachable, the top operation of every expression for t is $+$, $-$ or $\div$.*

Proof. A top multiplication combines $x \geq y \geq 2$ (Definition 3.1 forbids multiplying by 1), and xy is composite. □

Primes miss at 3.23% against 1.80% for composite targets. Taking the final multiplication away from composite targets as well raises their rate to 3.14%, so the lost route explains most, but not all, of the gap. The rest seems to follow the size of the largest prime factor: composite

targets whose largest prime factor exceeds 50 miss at 2.8%, close to the primes. We state the explanation as a conjecture: a target is hard roughly as far as it fails to factor into values that small sub-pools reach easily. Multiples of 25 are the easiest targets of all, missed 0.31% of the time.

13 Cats does Countdown

The game analysed so far deals one or two large numbers, the rule this project started with. On television, in *Countdown* and its comedy spin-off *8 Out of 10 Cats Does Countdown*, the contestant chooses any number of large numbers from none to four. Two special matches in 2010 went further. For Episode S26 (15 March 2010, Kirk Bevins against Chris Davies) and Episode S29 (16 August 2010, Charlie Reams against Junaid Mubeen) the large cards were swapped for the "nasty" numbers 12, 37, 62 and 87 [2]. We call the full choice of $\ell \in \{0, \ldots, 4\}$ large numbers *cats* mode and the same choice with the nasty cards *nasty-cats* mode. The solvers need no change, since they accept any pool, and every program now deals both modes on request (`countdown cats` or `countdown nasty`, optionally with the number of large numbers). The new pools are 2,850 with no large numbers, 840 with three and 55 with four, for 13,243 in all.

Large numbers ℓ	Pools	Standard cards	Nasty cards	Largest value
0	2,850	16.12%	16.12%	518,400
1	5,808	2.25%	3.15%	6,480,000
2	3,690	1.80%	0.78%	60,750,000
3	840	5.27%	0.41%	337,500,000
4	55	9.54%	0.45%	937,500,000
ℓ uniform on $0, \ldots, 4$		6.99%	4.18%	

Table 8: Probability of no exact solution given the number of large numbers chosen, with the target uniform and the cards dealt at random. The largest value is for the standard cards; with the nasty cards it is at most 239,493,600. All figures come from the same exhaustive computation as Theorem 12.1, in which the checker and the C solver agree on every new instance.

Table 8 gives the results, each an exact fraction rounded for display.

Six small numbers is the cruellest choice. With no large numbers 16.1% of rounds have no exact solution, eight times the standard game's rate. Here the range really does run out: 184 of the 2,850 pools cannot reach 999, and the pool $1, 1, 2, 2, 3, 3$, whose largest value is 81, reaches no target at all. The worst miss is 918 away.

Two large is the safest choice. With the standard cards, one or two large numbers are nearly equally good, and three or four are worse, 5.3% and 9.5%. Fewer small numbers leave less room to adjust a large product: with four large, the pool $1, 1, 25, 50, 75, 100$ misses 575 targets.

The nasty numbers are kinder. A fan wiki claims that the nasty cards make rounds *more* likely to be solvable [2]. That holds for two, three and four large numbers, dramatically for four (0.45% against 9.54%), and on every average we tried, but it fails for one large number, where the miss rate rises from 2.25% to 3.15%. The culprit is 12, a weak single large number: the pool $1, 1, 2, 2, 3, 12$ misses 792 targets. We conjecture that the standard cards are poor together because 25, 50, 75 and 100 are all multiples of 25, so they make fewer distinct combinations, while 12, 37, 62 and 87 share little.

The bounds no longer hold. The remark after Proposition 3.5 that values stay below 2^{27} is true only for one or two large numbers. With four standard cards the pool $10, 10, 25, 50, 75, 100$ reaches 937,500,000, and 73 pools exceed 2^{27}. The bound itself, $\prod_i (p_i + 1) - 1$, stays below $101 \cdot 76 \cdot 51 \cdot 26 \cdot 11^2 < 2^{31}$, so values still fit in a signed 32-bit integer and the reciprocal division test of Lemma 7.3 still applies, but with no room to spare. Every solver therefore has to be re-checked on cats rounds.

Every solver re-checked. Two further benchmark sets of 10,000 rounds each, one per mode, deal ℓ uniformly from 0 to 4. Their reference answers come from the C solver and agree with the Rust version on every round; each answer's distance from the target also matches the exhaustive checker. Table 9 gives the results for a selection of solvers; `PERFORMANCE.md` lists all of them. Every CPU solver gives the reference answer on every round in both modes, apart from Python's usual equally good ties. The GPU solvers first packed each value into 24 bits, enough for the standard game, and refused larger pools rather than answer wrongly. Cats values below the whole pool reach 93,750,000, which needs 27 bits. PyTorch now gives every value 27 bits, at no cost on the standard set. CUDA keeps its 24-bit shared-memory tables, since subsets two smaller than the pool stay below 2^{24}, and sends the rounds that need it to a second kernel with 64-bit keys for the larger subsets. Both now give the reference answer on every round, and the transformer, which proves its proposals with the PyTorch solver, gives no wrong answers. The lookup table covers only standard pools and passes every other round to the Python solver.

Solver	Standard	Cats	Nasty-cats	Result in both modes
Rust + CUDA	0.000442	0.000626	0.000475	all identical
PyTorch	0.0156	0.0229	0.0158	all identical
PyTorch with transformer	0.0354	0.0441	0.0211	equally good ties only
ISPC	0.0115	0.067	0.0151	all identical
x86-64 assembly	0.0218	0.044	0.0220	all identical
C	0.0208	0.055	0.0209	all identical
Erlang	0.0575	0.103	0.0641	all identical
Elixir	0.0576	0.106	0.0669	all identical
Rust	0.0451	0.0967	0.0454	all identical
Go	0.0330	0.112	0.0353	all identical
Java	0.0391	0.145	0.0440	all identical
Haskell	0.0360	0.332	0.0409	all identical
Ruby	0.494	1.06	0.494	all identical
Python	7.23	7.33	6.83	equally good ties only
Lookup table	0.00069	4.68	6.74	ties from its Python fallback

Table 9: Solve time in milliseconds per round on the standard, cats and nasty-cats benchmark sets. The cats and nasty-cats times are the best of three runs for the solvers changed for cats and of one run for the others, and so are rougher than those of Table 4.

Four large numbers are slow for the fastest solvers. Cats rounds first took C 5.6 times as long as standard ones, and the cost was almost all in rounds with four large numbers: C took 0.018 ms with one large number, 0.059 ms with three and 0.46 ms with four. Like many of the compiled ports, C marks the values each subset has made in a bitset sized by that subset's largest possible value (Section 7.2, "Duplicates by bitset"), and with four large numbers that bitset reaches 11.7 MB. Scanning it for ranks and clearing it cost more than the search did. Erlang and Elixir sort each subset's values instead and slow down by only 1.8 times. C now avoids most of the cost: it radix sorts any subset whose bitset has more than 64 words per value and clears only the words that subset touched. That took four-large rounds from 0.44 to 0.16 ms and the cats set from 0.114 to 0.055 ms, with no change on the standard set; Half of

what remains is zeroing the bitset once per solve. The same path was ported to the 22 other solvers with a full-range rank pass, and most of them halved their cats time with no change on the standard set; Table 9 shows the new figures. Hand-written assembly gained most and is now the fastest single-core solver on cats, at 0.044 ms. The nasty cards' largest value is a quarter of the standard cards', and most compiled solvers run nasty-cats rounds within 35% of their standard time. Rust was the exception, at first taking 0.126 ms on nasty-cats rounds, 88% of it in `memset`. It allocated a bitset sized for the whole pool on every solve. With four standard cards that is about 150 MB, and glibc maps that much straight from the operating system as lazily zeroed pages. With four nasty cards it is 7–30 MB, below glibc's mapping threshold, so it came from the heap and was cleared in full. Growing the bitset by subset size, as C does, brought Rust to 0.045 ms on nasty-cats, 0.097 on cats and 0.045 on the standard set. A solver for every mode would choose between the bitset and sorting by the size of the bound.

14 Machine-checked guarantees

Theorem 12.1 and Table 8 rest on programs agreeing. This section re-derives the key claims with four tools of different kinds: a proof assistant, a SAT solver with proof checking, an SMT solver and a bounded model checker of the C source. Table 10 lists which tool supports which claim and how far each result can be trusted. The scripts, logs and full results are in the repository's `solvability/` and `verification/` directories.

Claim	Tool	Trusted base and coverage
Decision procedure is sound and complete	Lean 4	Kernel only
Value bound (Proposition 3.5)	Lean 4	Kernel only
Worst pool: 653 misses, maximum 675	Lean 4 `decide +kernel`	Kernel only
Every count and probability in Theorem 12.1 and Table 8	Lean 4 `native_decide`	Kernel plus the Lean compiler, in four named facts
No solution, pilot instances	CaDiCaL with LRAT	Proofs checked by the verified `cake_lpr`; the encoder is unverified
No solution, pilot instances	Lean 4 `bv_decide`	Kernel plus the Lean compiler; encoding proved complete in the kernel
No solution, pilot instances	Z3, unbounded integers	Z3 trusted; every SAT answer re-evaluated
Largest cats values	Z3	Z3 trusted
Proposition 12.3, one-node steps of Propositions 12.2 and 3.5	Z3	Z3 trusted; trees of up to 3 leaves in full
Reciprocal division (Lemma 7.3)	CBMC, Z3 and an exhaustive run	Hardware exhaustive for every 32-bit dividend; symbolic proofs of cells only
Binary search, kernels and cats safety of `solver.c`	CBMC	Bounded, on the solver's own source

Table 10: Which claim is checked by which tool. "Pilot instances" are the 653 misses of the worst pool $1, 1, 2, 2, 3, 25$, target 967 on all 2,058 pools that miss it and the 575 misses of the cats pool $1, 1, 25, 50, 75, 100$.

Lean. A Lean 4 development with no external library defines expressions, their evaluation and legality (each dealt card used at most once) and the predicate Solvable(p, t) directly from the

rules, with no reference to any algorithm. It then defines the subset recursion of Theorem 3.4 as a program and proves, in the kernel, that a target is in its output exactly when it is solvable. Because the program is proved correct for every pool, running it is a proof. Running it in the kernel is fast enough for one pool: the worst pool's 653 misses, its maximum of 675 and its smallest miss, 179, are checked with no trust beyond the kernel. Running it over all 23,636 pools of the three modes needs compiled code, so the counts and probabilities use `native_decide`, which adds the Lean compiler to the trusted base. The development confines this to four named facts, from which every count of Theorem 12.1 and every rate of Table 8 follows in the kernel. Its per-pool miss lists equal the exhaustive study's for every pool of every mode. A clean build takes about 95 seconds.

SAT. To give the "no solution" half a certificate that a small checker can verify, each unsolvable instance is encoded as a bit-vector formula: a straight-line program of at most five steps over the six numbers, each step adding, subtracting, multiplying or exactly dividing two unused values, ending at the target. The width is the least w with $\prod_i (p_i + 1) \leq 2^w$, safe by Proposition 3.5, and a whole miss set can be given as the target, so one unsatisfiability proof covers every miss of a pool. CaDiCaL proves each formula unsatisfiable and writes an LRAT proof, which the CakeML-verified checker `cake_lpr` accepts. A second route proves in Lean that the encoding is complete, so that unsatisfiability implies $\neg\,\mathrm{Solvable}(p, t)$, and discharges each formula with `bv_decide`, which runs a verified bit-blaster and LRAT checker as compiled code. Both routes agree with the exhaustive study on all pilot instances, about 4,200 in all, and every satisfiable formula decodes to an expression the evaluator accepts. Proofs take about 1 second for the worst pool, 4 seconds for target 967 and 19 seconds for the cats pool, whose 26-bit values make multiplication much harder to encode. Extrapolated, every standard miss would take about 56 core-hours by either route and the three modes together about a core-day, so a full certificate for Theorem 12.1 is within reach.

Z3. Z3 gives a third, unbounded encoding of "no solution" in integer arithmetic. It agrees on the worst pool (653 unsatisfiable, 247 satisfiable), on target 967 for 100 sampled pools and on the 517 misses of the cats pool it reached in the time allowed. It proves that 937,500,000 and 239,493,600 are the largest cats and nasty-cats values, that no prime target in range is a product xy with $x \geq y \geq 2$ (Proposition 12.3), and the one-node inequalities from which Propositions 12.2 and 3.5 follow by induction; the induction itself is on paper. Z3's unsatisfiability proofs cannot be checked independently, so these results trust Z3.

CBMC. The bounded model checker CBMC checks properties of the C solver by including its source unchanged. It proves the binary search correct for symbolic sorted lists of up to 32 entries, the result-emitting kernel correct for 24 appends, and, for the full ranges of cats and nasty-cats values, that the partner search, the exactness check and the bitset indexing cannot overflow or go out of bounds. The solver's building of sorted value lists is too heap-heavy for CBMC beyond the smallest case and was instead tested exhaustively on 5.4 million pairs of small lists. Every cats and nasty-cats pool was also solved by a build with address, undefined-behaviour and integer sanitisers, with no reports. The solver would only break on repeated large numbers, which no mode deals.

The reciprocal test. Lemma 7.3 resisted symbolic proof: CBMC and Z3 both time out on its floating-point multiplication over the full range, proving only cells of it. A direct run on the hardware settles it instead: for every divisor $2 \leq y < 2^{32}$ and every multiple of y below 2^{32}, 91,633,733,620 cases in 33 seconds, the test gives the exact quotient, and a single-precision mutant fails 11.4 billion of them. So for $y \geq 2$ the lemma holds for every 32-bit dividend, beyond

the 2^{31} the proof needs. For $y = 1$ the bound is tight, failing first at $x = 2^{31} + 1$, which is why the solvers never divide by 1.

15 How the implementations were written

Every implementation, benchmark script and this paper were written by one large language model, Anthropic's Claude Opus 5.5 (model identifier `claude-opus-5-5`), working in the Claude Code command-line tool (versions 2.1.291–2.1.296) between 6 and 10 October 2026. A person set the goals, chose the languages and set rules such as no state carried between solves; the model wrote and ran all the code and proposed the analyses. The early languages were written in the main conversation. The later languages, the parallel versions and the ISPC solver were each handed to a subagent, a separate instance of the same model, given a written brief and run in parallel with others on separate cores.

By the repository's `bench/agent_cost.py`, the work took about 2.75 million output tokens and 460 million input tokens, most of them cached context read again on each call. About 17% of the output went on general work such as this paper and the benchmark runs. The rest is charged to individual solutions in `PERFORMANCE.md`. Ports of an existing optimised design were cheapest, under 21,000 output tokens each. Work that needed exploration or debugging cost the most: ISPC took about 258,000 tokens, from about 60 measured variants, and the two parallel BEAM versions about 160,000 each.

We record this so that the work can be repeated with later models. A fair comparison would give a new model the same benchmark set, reference answers, harnesses and rules, and measure the same quantities: solve time pinned to one core, byte-identical answers by `bench/check.py`, lines of code by `bench/loc.py`, and tokens and time spent. Timings should be re-measured on the same machine, since the results here show session-to-session variation of about 10%. Two parts of this record are themselves estimates: subagent transcripts store token usage before a response finishes, so subagent output is estimated from the length of what was written, and time is attributed per response with a 15-minute cap.

References

[1] G. Hutton. The countdown problem. *Journal of Functional Programming*, 12(6):609–616, 2002.

[2] The Countdown wiki (Apterous): Episode S26, Episode S29 and "Nasty". `https://wiki.apterous.org/Episode_S26`, `https://wiki.apterous.org/Episode_S29`, `https://wiki.apterous.org/Nasty`.

[3] The Countdown solvers, benchmarks and this paper's source. `https://github.com/ZippyManaic/countdown`, 2026.